What is TinyML? An Introduction to Tiny Machine Learning

Learn about TinyML, its applications and benefits, and how you can get started with this emerging field of machine learning.

Machine learning models play a prominent role in our daily lives – whether we know it or not. Throughout the course of a typical day, the odds are that you will interact with some machine learning model since they have permeated almost all the digital products we interact with; for example, social media services, virtual personal assistance, search engines, and spam filtering by your email hosting service.

Despite the many instances of **machine learning** in daily life, there are still several areas the technology has failed to reach. The cause? Many machine learning models, especially state-of-the-art (SOTA) architectures, require significant resources. This demand for high-performance computing power has confined several machine learning applications to the cloud – on-demand computer system resource provider.

In addition to these models being computationally expensive to train, running inference on them is often quite expensive too. If machine learning is to expand its reach and penetrate additional domains, a solution that allows machine learning

models to run inference on smaller, more resource-constrained devices is required. The pursuit of this solution is what has led to the subfield of machine learning called Tiny Machine Learning (TinyML).

In this article, we will:

- Define TinyML and its benefits
- Cover some applications of TinyML
- Discuss the workflow requirements involved for TinyML applications.

What is TinyML?

"*Neural networks are also called artificial neural networks (ANNs). The architecture forms the foundation of **deep learning**, which is merely a subset of machine learning concerned with algorithms that take inspiration from the structure and function of the human brain. Put simply, neural networks form the basis of architectures that mimic how biological neurons signal to one another.*"
Source; **PyTorch Tutorial: Building a Simple Neural Network**

Machine learning is a subfield of **artificial intelligence** that provides a set of algorithms. These algorithms allow machines to learn patterns and trends from available historical data to predict previously known outcomes on the same data. However, the main goal is to use the trained models to generalize their inferences beyond the training data set, improving the accuracy of their predictions without being explicitly programmed.

One such algorithm used for these tasks is neural networks. Neural networks belong to a subfield of machine learning known as deep learning, which consists of models that are typically more expensive to train than machine learning models. You can learn more about **building neural network models in R** in a separate tutorial.

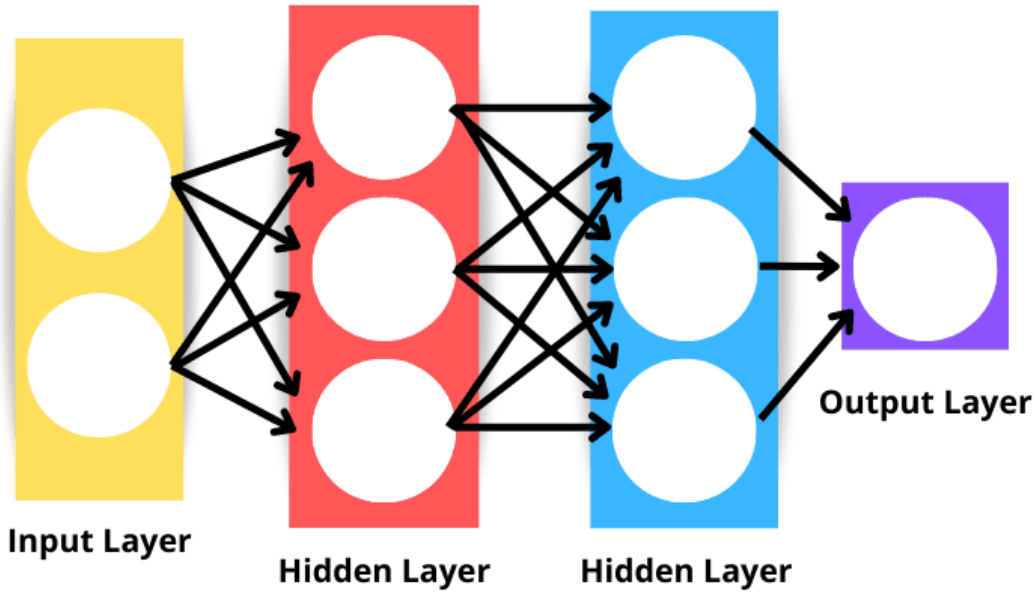

Figure 1. A visualization of a three-layer neural network

According to [tinyml.org](), "*Tiny machine learning is broadly defined as a fast-growing field of machine learning technologies and applications including hardware, algorithms, and software capable of performing on-device sensor data analytics at extremely low power, typically in the mW range and below, and hence enabling a variety of always-on use-cases and targeting battery operated devices.*"

TinyML's growth in recent years has largely been attributed to the development of the hardware and software ecosystems that support it. Since the techniques can be implemented in low-energy systems (i.e., sensors, microcontrollers, etc.), machine learning can be brought to the edge in an extreme way, enabling such applications to perform with real-time responsivity. In essence, the idea is to enable machine learning practitioners to do more with less.

But why is this so important? Let's take a look at why TinyML is appealing.

The benefits of TinyML

1. **Latency**: The data does not need to be transferred to a server for inference because the model operates on edge devices. Data transfers typically take time, which causes a slight delay. Removing this requirement decreases latency.
2. **Energy savings**: Microcontrollers need a very small amount of power, which enables them to operate for long periods without

needing to be charged. On top of that, extensive server infrastructure is not required as no information transfer occurs: the result is energy, resource, and cost savings.

3. **Reduced bandwidth**: Little to no internet connectivity is required for inference. There are on-device sensors that capture data and process it on the device. This means there is no raw sensor data constantly being delivered to the server.
4. **Data privacy**: Your data is not kept on servers because the model runs on the edge. No transfer of information to servers increases the guarantee of data privacy.

Use Cases: How is TinyML being used?

The applications of TinyML spread across a wide range of sectors, notably those dependent on internet of things (IoT) networks and data – The Internet of Things (IoT) is basically a network of physical items embedded with sensors, software, and other technologies that connect to and exchange data with other devices and systems over the internet.

Computer vision, visual wake words, keyword spotters, predictive maintenance, gesture recognition, industrial machine maintenance, etc., are all common TinyML use cases. Let's also take a look at some industries where TinyML has been used to power applications:

Agriculture

Real-time agriculture and livestock data can be monitored and collected using TinyML devices. The Swedish edge AI product business **Imagimob** has created a development platform for machine learning on edge devices. Fifty-five organizations from throughout the European Union have collaborated with Imagimob to learn how TinyML can offer efficient management of crops and livestock.

Industrial predictive maintenance

TinyML can be deployed on low-powered devices to continuously monitor machines for malfunctions and predict issues before they happen; this type of application boasts the potential to help businesses reduce costs that often arise from faulty machines.

A prime example of predictive maintenance is **Ping Services**. They developed a monitoring device to continuously monitor the acoustic signature of wind turbine blades to detect and notify of any change or damage. According to Ping's website, "continuous monitoring operators can give a timely response to blade damage, reducing maintenance costs, failure risks, and downtime, as well as improving wind turbine performance and efficiency."

Customer Experience

Personalization is a key marketing tool that customers demand as their expectations rise. The idea is for businesses to understand their customers better and target them with ads and messages that resonate with their behavior. Deploying edge TinyML applications enable businesses to comprehend user contexts, including their behavior.

Workflow Requirements

Many tools and architectures deployed in traditional machine learning workflows are used when building edge-device applications. The main difference is that TinyML allows these models to perform various functions on smaller devices.

Tensorflow Lite for microcontrollers (TF Lite Micro) is one of the most popular frameworks for machine learning on edge devices; it was specifically designed for the task of implementing machine learning on embedded systems with only a few kilobytes of memory.

Python is often the preferred language for building machine learning models. However, TensorFlow Lite makes it possible to easily develop models in C, C++, or Java and deploy them without connecting to the internet.

From a hardware perspective, a supported microcontroller board is required o get started with TinyML in TF Lite; the library currently supports the following microcontrollers:

- Arduino Nano 33 BLE Sense
- SparkFun Edge
- STM32F746 Discovery kit
- Adafruit EdgeBadge
- Adafruit TensorFlow Lite for Microcontrollers Kit
- Adafruit Circuit Playground Bluefruit
- Espressif ESP32-DevKitC
- Espressif ESP-EYE
- Wio Terminal: ATSAMD51
- Himax WE-I Plus EVB Endpoint AI Development Board
- Synopsys DesignWare ARC EM Software Development Platform
- Sony Spresense

With the support of TinyML, it is possible to increase the intelligence of billions of devices we use every day, like home appliances and IoT gadgets, without spending a fortune on expensive hardware or dependable internet connections, which are frequently constrained by bandwidth and power and produce significant latency.

Learning Resources

- **TinyML Foundation**
- **Tiny ML: Machine Learning with Tensorflow Lite on Arduino and Ultra-Low-Power Microcontrollers** (Book)
- **Embedded Machine Learning on Edge Devices (Podcast)**
- **Understanding Machine Learning**
- **Introduction to Deep Learning in Python**
- **Deep Learning Tutorial**

Wrap-up

TinyML has been gaining traction across various industries in recent years due to the development of hardware and software ecosystems that support it. The tool has made it possible to implement machine learning models in low-energy systems, like microcontrollers, which opens the door to various new opportunities. Low latency, energy savings, data privacy, and no connection dependencies are some of the factors that make TinyML so appealing to developers seeking to develop applications for internet of things (IoT) devices.